

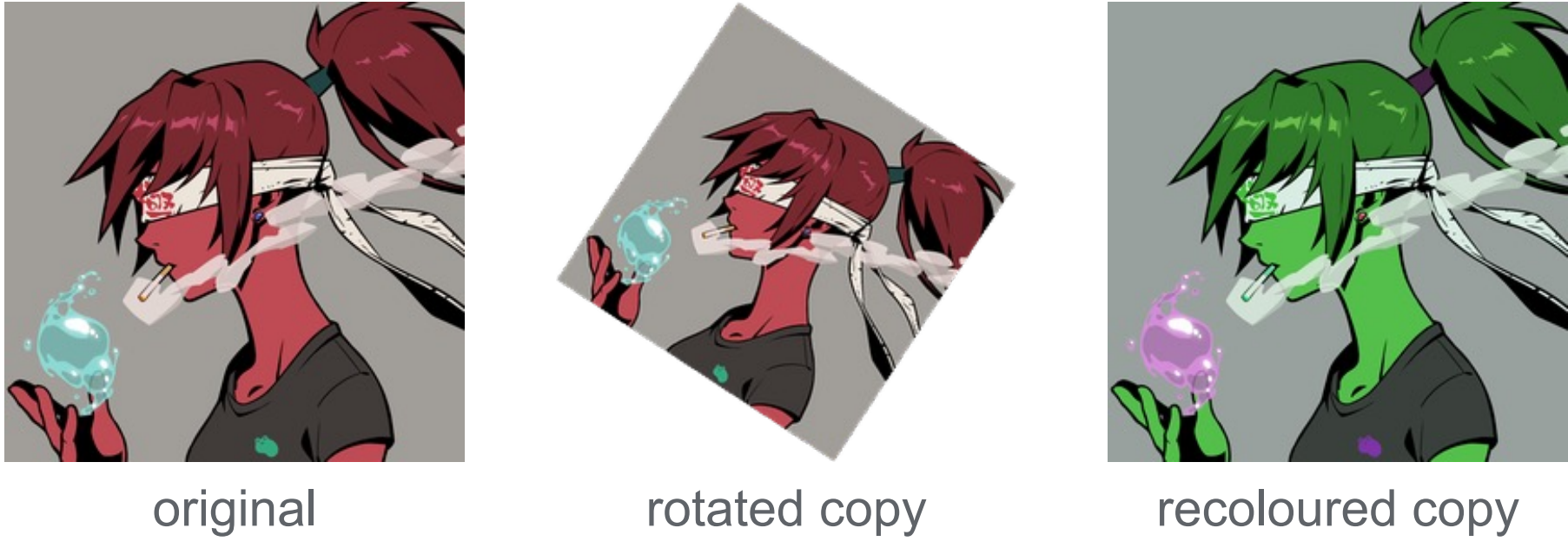
Detecting Copied NFT Images

Replacing a broken signal in an NFT copymint detector

Rabea Yassin · Ahmad Nassar Instructor: Eran Tavor

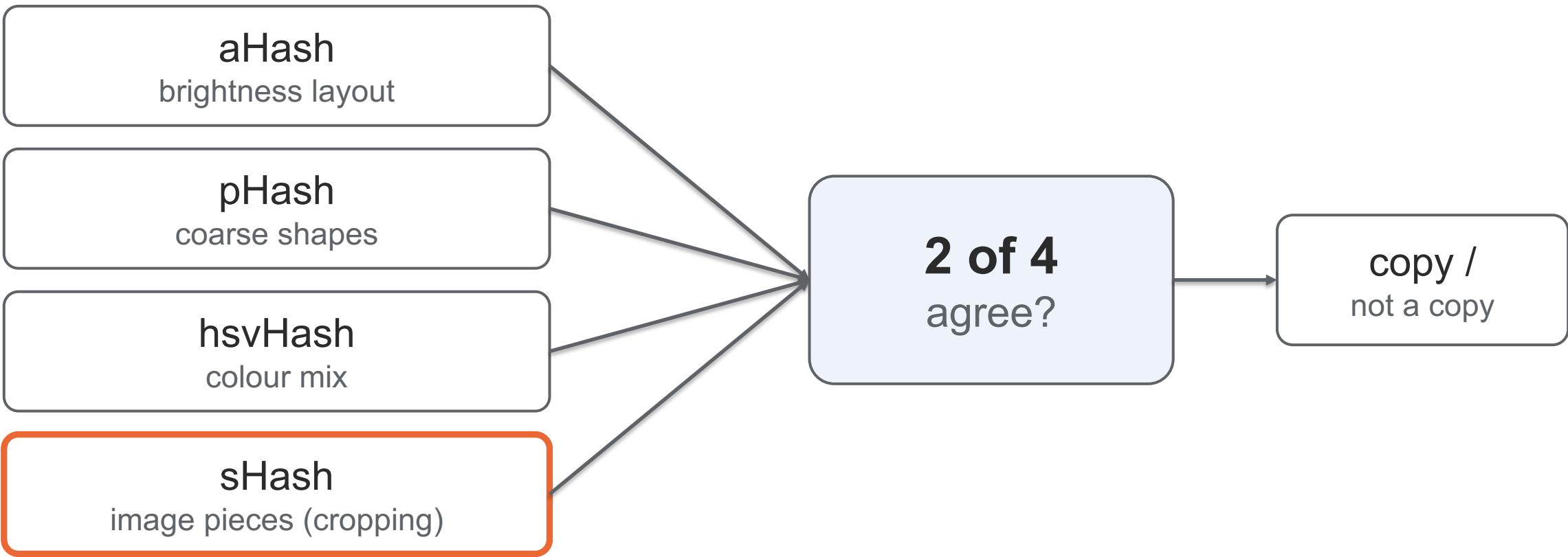
The problem: copymints

A copymint takes someone's NFT artwork, edits it a little, and mints it as new. Exact hashing (SHA-256) can't see it: one changed pixel gives an unrelated digest. Detection needs fingerprints of **what the image looks like**.



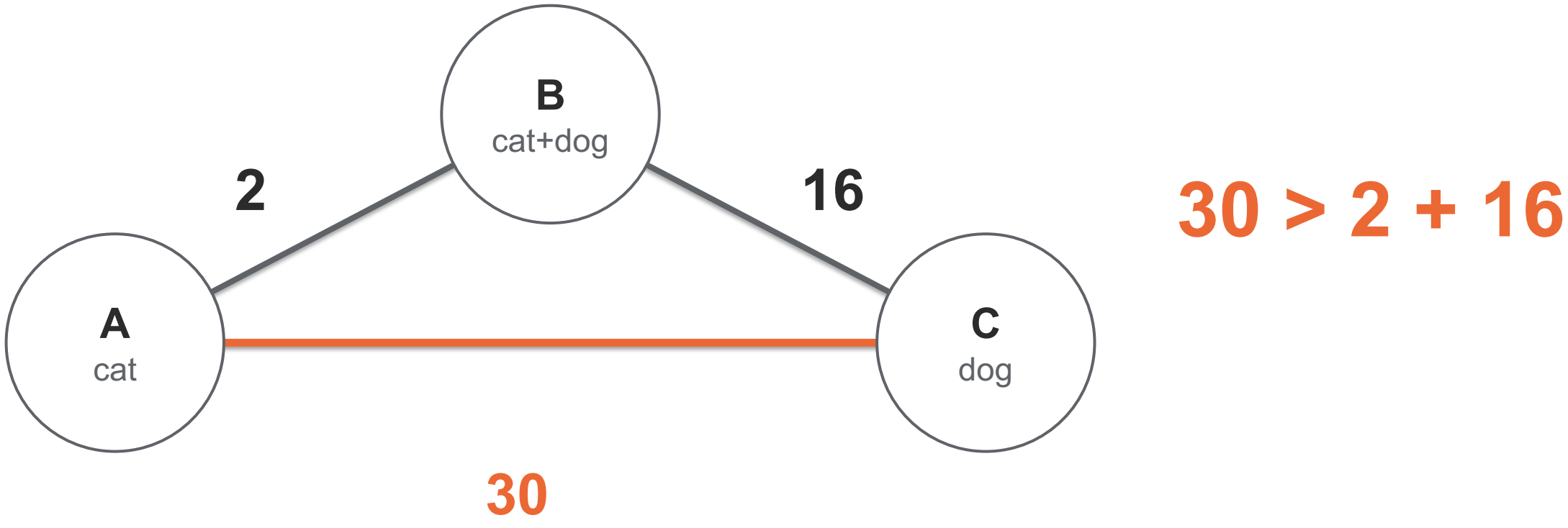
What the researchers built

Kotzer et al. compute four perceptual hashes per image, search each in its own BK-tree, and call it a copy when **at least two agree**.



What we found: sHash isn't a distance

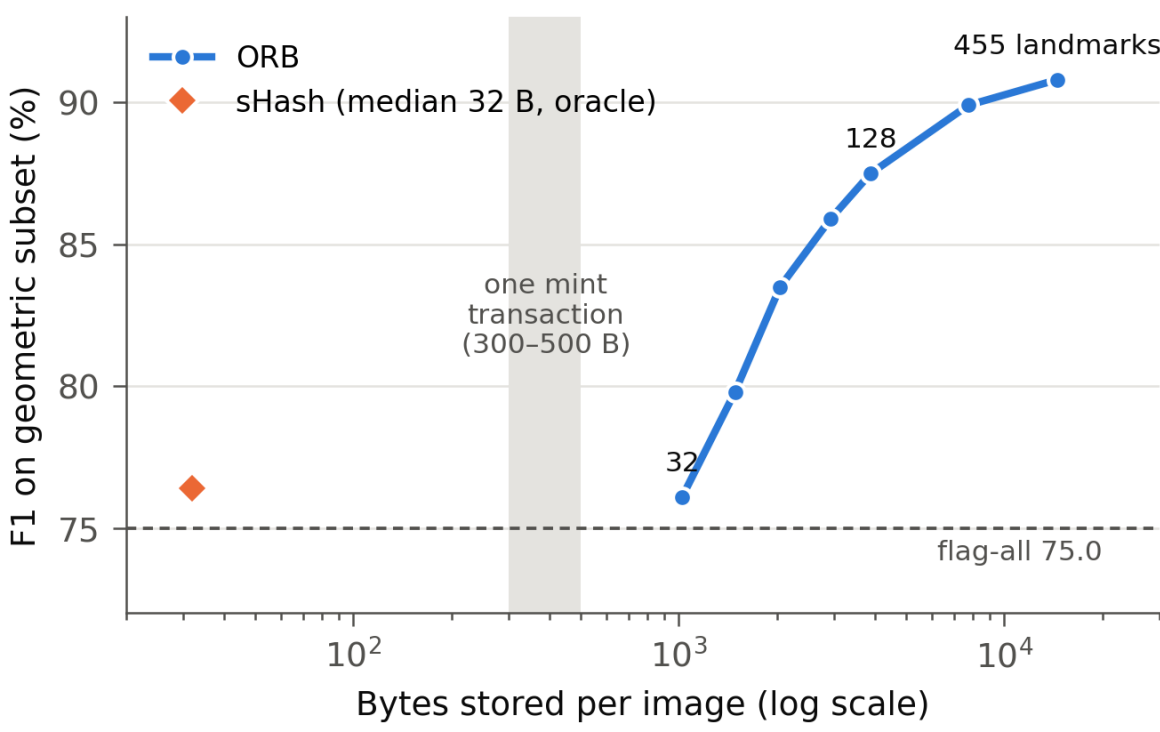
sHash fingerprints each **piece** of an image and compares the lists **one way**. In the data the authors shared with us, the reported distance matches one direction on all 1,802 pairs, and the other on only 240.



A search tree assumes the direct route is never longer than the detour. With sHash it can be, so the tree **skips a branch holding a real copy** and nothing looks wrong.

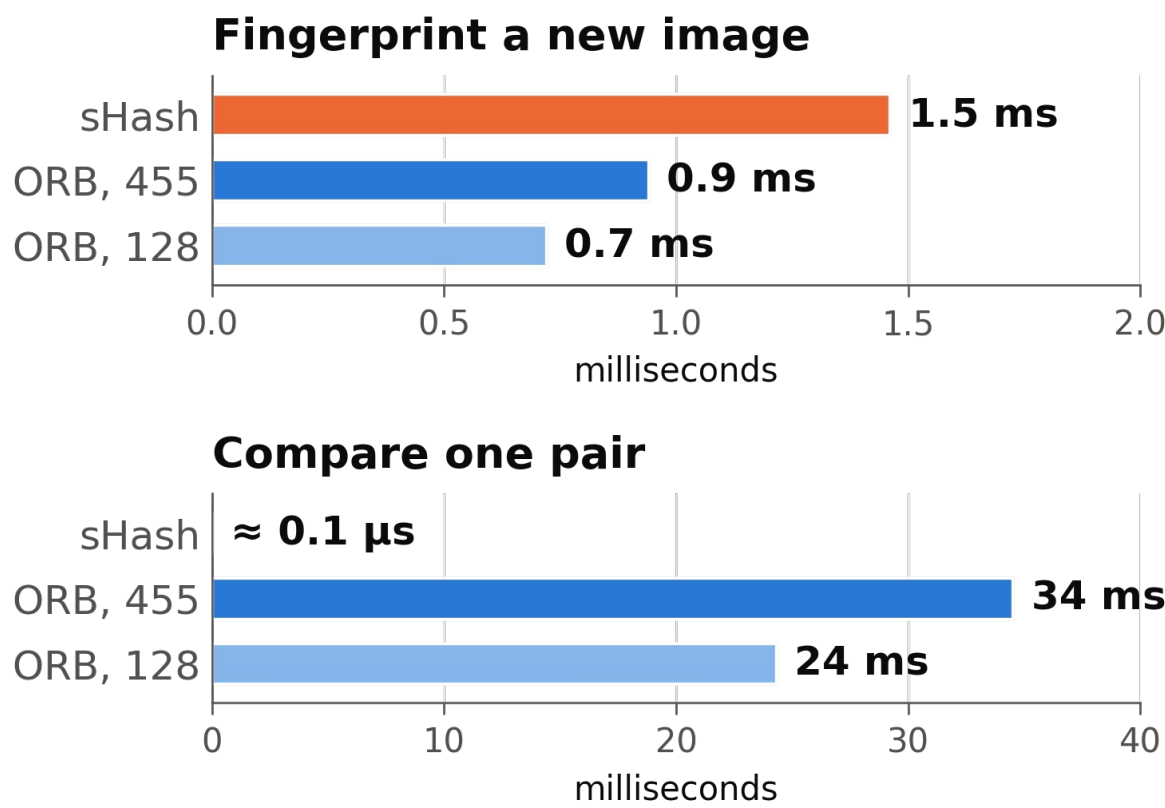
What ORB costs: space and time

Space: too big for the transaction



The paper stores its four hashes (**≈56 B**) in each mint transaction (300–500 B). ORB needs **14,560 B**, about 36 transactions; at 32 landmarks it only ties sHash. **No size both fits and wins.**

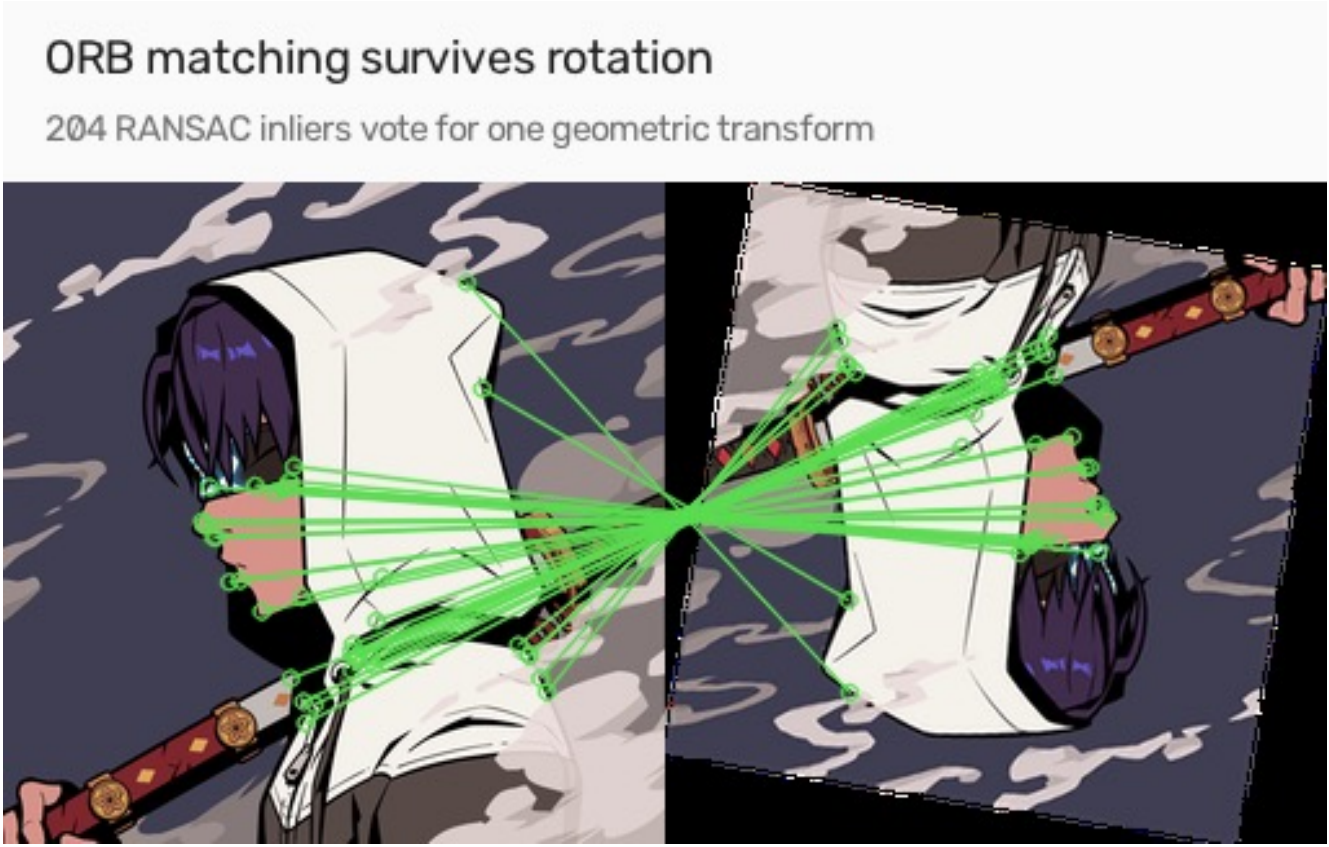
Time: comparing is what's slow



Making a fingerprint costs about the same. **Comparing a pair takes ≈34 ms for ORB, ≈0.1 μs for sHash**; shrinking ORB to 128 landmarks still leaves 24 ms. ORB needs an index such as LSH.

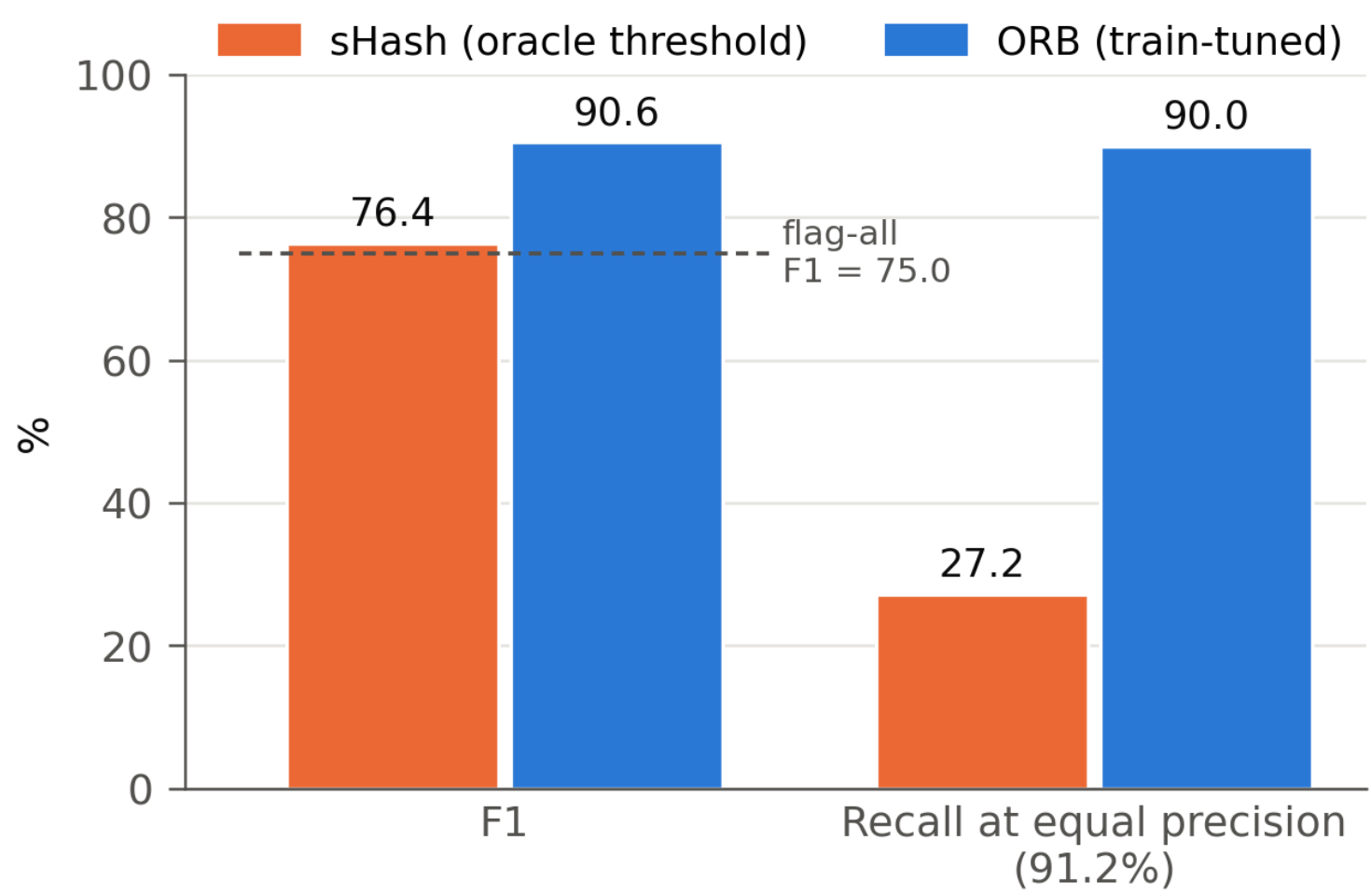
455 / 128: landmarks kept. Python library, as used: 68 ms, 11 μs.

Our replacement: ORB feature matching



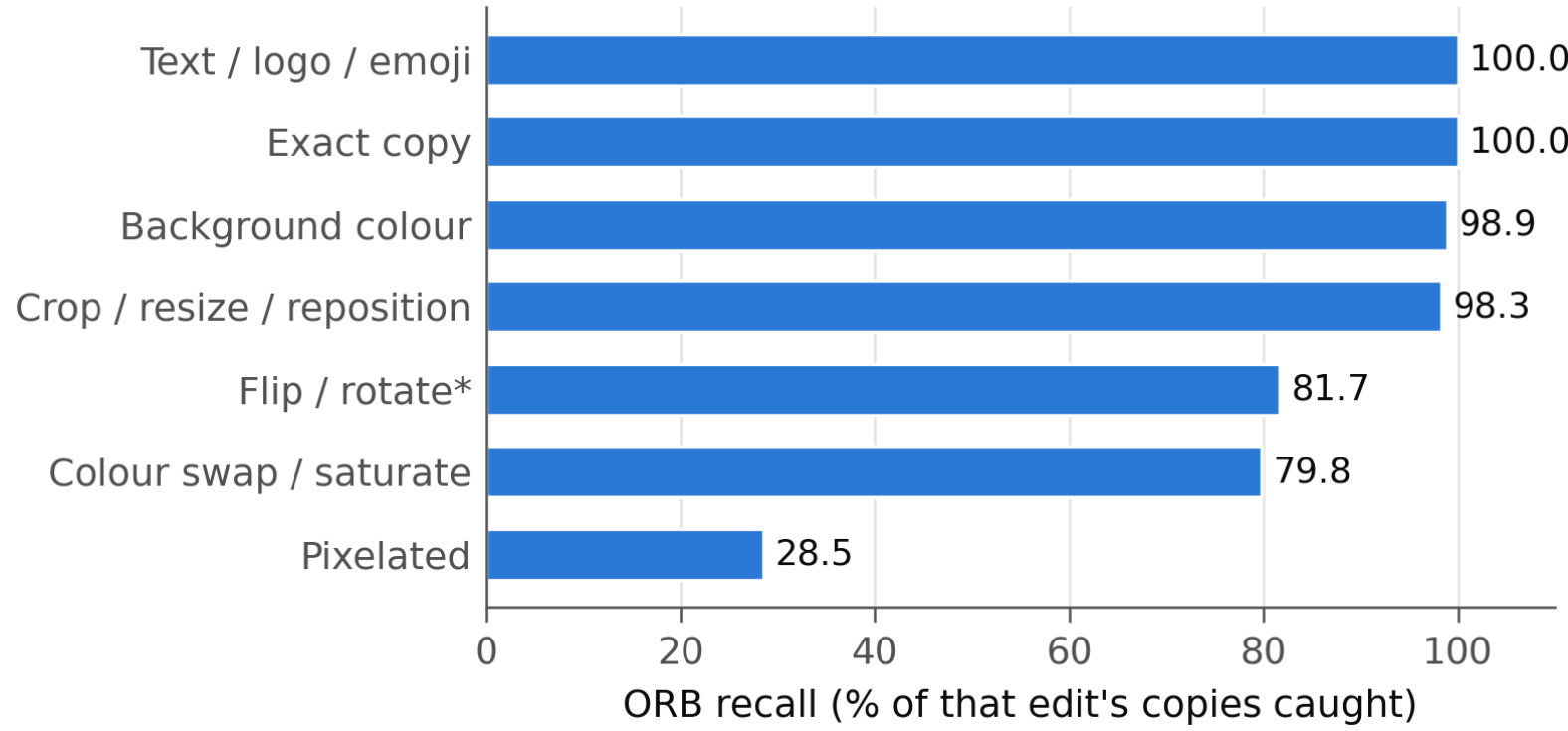
Find landmarks (corners) in both images, **match** them, and let **RANSAC** keep only the matches that agree on one transformation. Here 204 matches agree on one rotation. ORB replaces sHash; the other three hashes and the vote are untouched.

Result 1: +14.2 F1 on sHash's own job



On the crop and rotation copies sHash was built for. sHash was **allowed to pick its threshold on the test answers**; ORB's was set on training data. sHash keeps its recall by flagging **1,630 of 2,400** innocent images.

Result 2: it wins almost everywhere

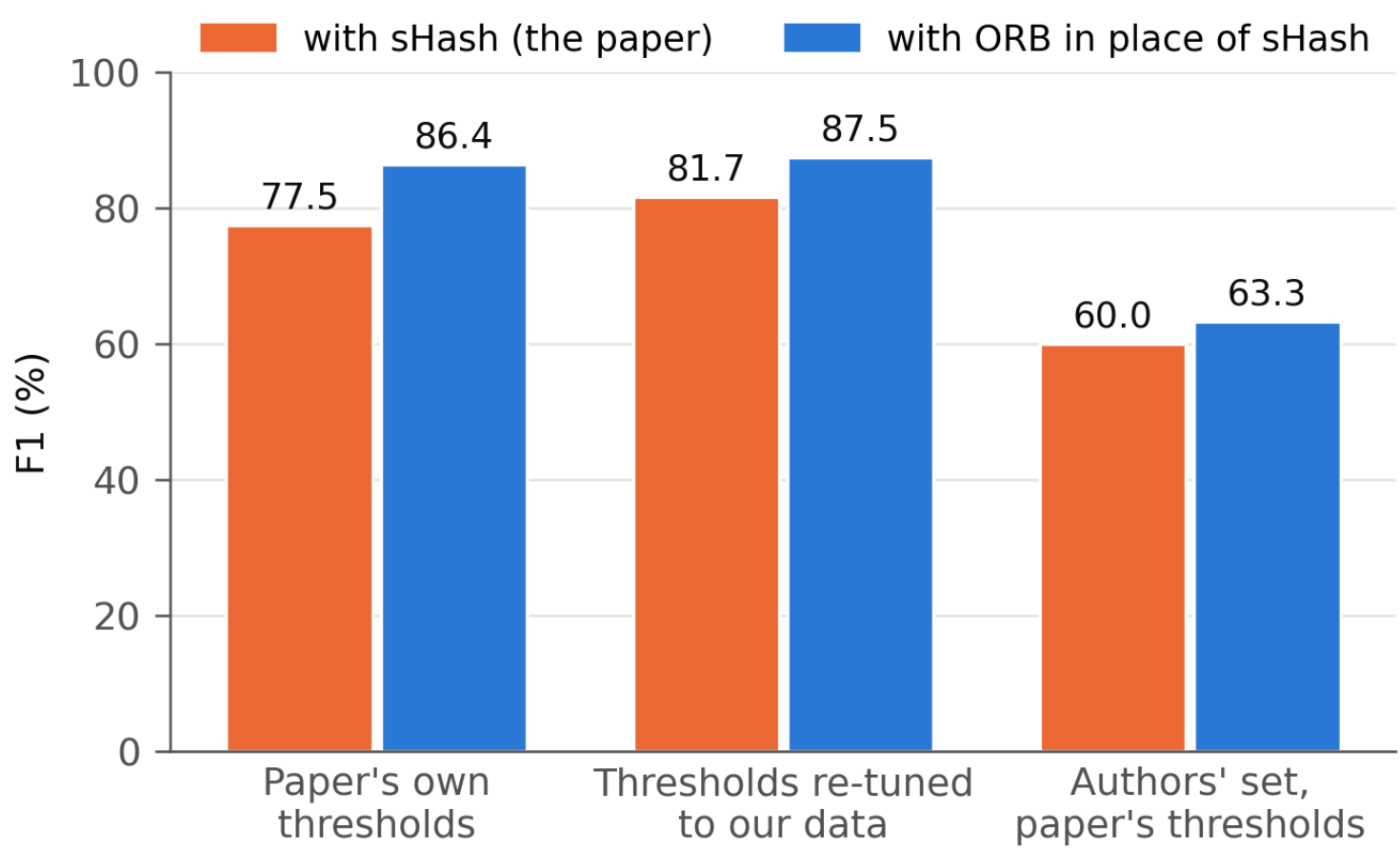


Share of each edit's copies ORB catches (precision 91.2%).

One clean failure: **pixelation** destroys the detail corners are made of.

*Always includes rotation; pure mirrors: 100%.

Result 3: +8.9 F1 in the whole detector



Change only sHash; keep the paper's own thresholds. At the same **98.1% precision**, recall rises from **64.1% to 77.2%**: ~1,500 more copies caught.

Our test set is 82.6% copies, so equal precision is the fair comparison.

What we learned

Check the maths before optimising it. We set out to make the search faster and found it was unsound.

How a signal is stored decides how it can be searched. A fingerprint fits a tree; a set of landmarks needs LSH, far more space, and ≈34 ms per pair instead of ≈0.1 μs.

A negative result is still a result. Our edit classifier predicts which signals an edit breaks (100% / 97.5%), but doesn't improve the detector: broken signals go silent rather than voting wrongly.

Next: keep ORB's landmarks off-chain with a small commitment on-chain. Letting the cheap hashes shortlist candidates for an off-chain ORB check is an untested idea.